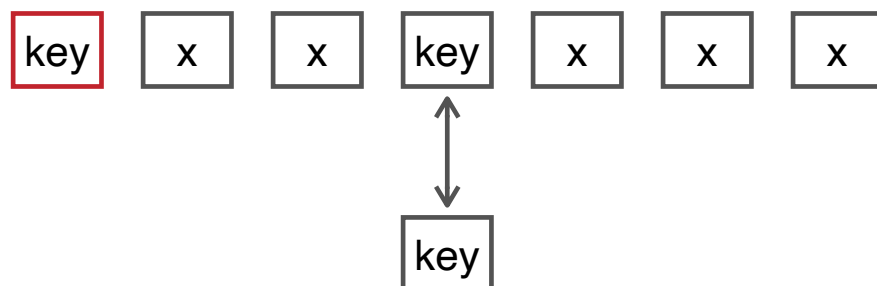




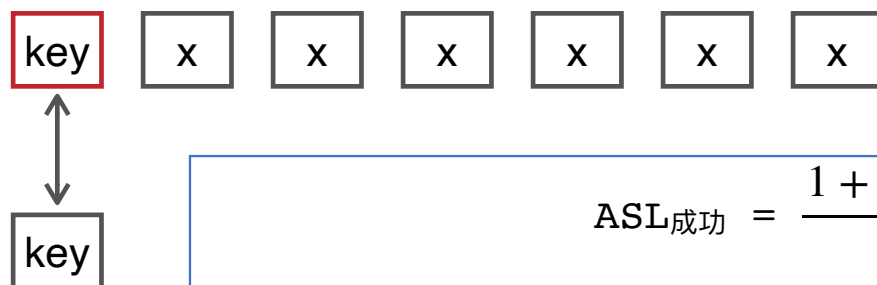


考点1 线性表的查找

①顺序查找



- ①顺序查找可以用于顺序/链式存储，且不要求元素有序
- ②算法简单，但ASL（平均查找长度）较大，查找效率不高
- ③当查找表规模较大时，不适合使用顺序查找



$$ASL_{成功} = \frac{1 + 2 + 3 + \dots + n}{n} = \frac{n + 1}{2} \approx \frac{n}{2}$$

$$ASL_{失败} = n + 1 \text{ (有哨兵)}、n \text{ (无哨兵)}、\frac{n}{2} + \frac{n}{n + 1} \text{ (有序优化)}$$

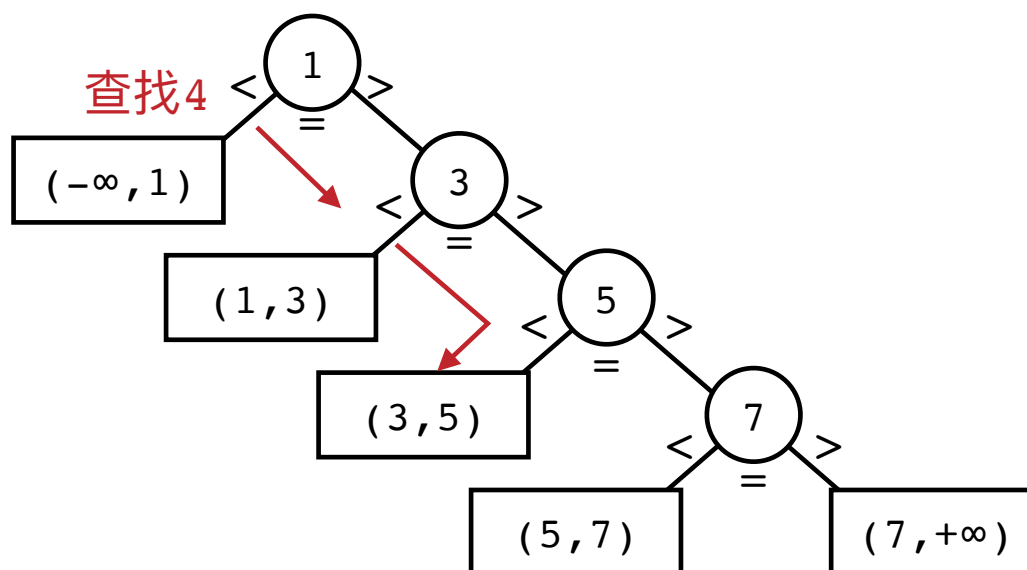


考点1 线性表的查找

①顺序查找

$$ASL_{成功} = \frac{1 + 2 + 3 + \dots + n}{n} = \frac{n + 1}{2} \approx \frac{n}{2}$$

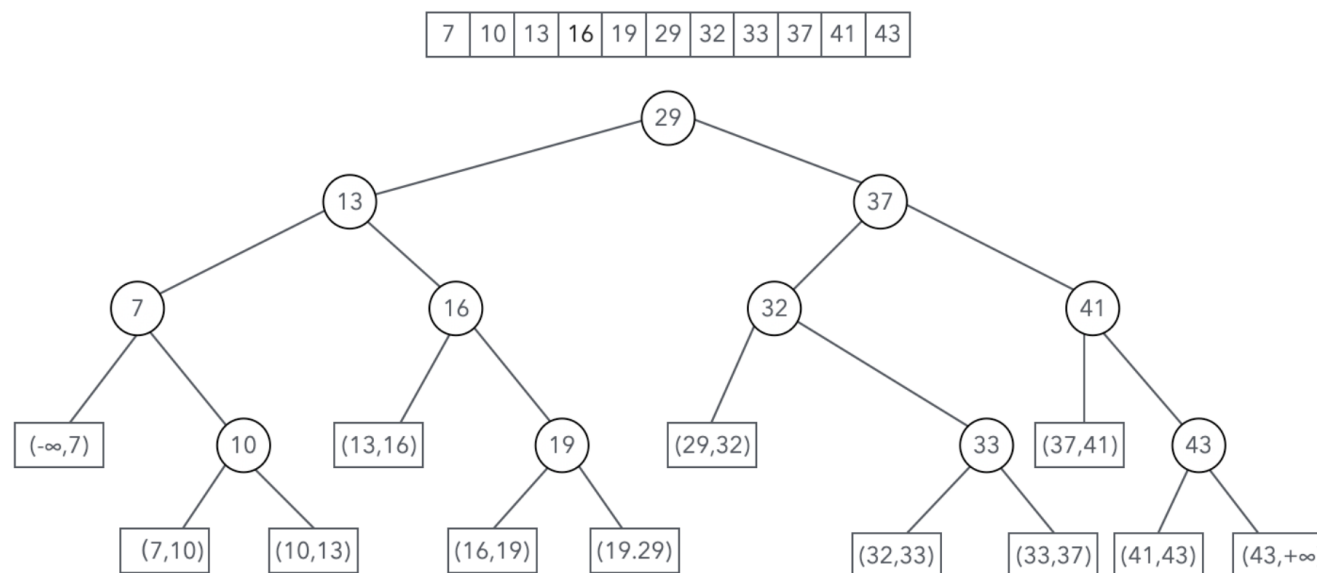
$$ASL_{失败} = n + 1 \text{ (有哨兵)}、n \text{ (无哨兵)}、\frac{n}{2} + \frac{n}{n + 1} \text{ (有序优化)}$$





考点1 线性表的查找

②折半查找



①折半查找只适用于顺序存储（需要借助随机访问特性来快速定位元素），且要求元素必须有序

②mid的选取必须保证整体一致，默认左右选左（向下取整）

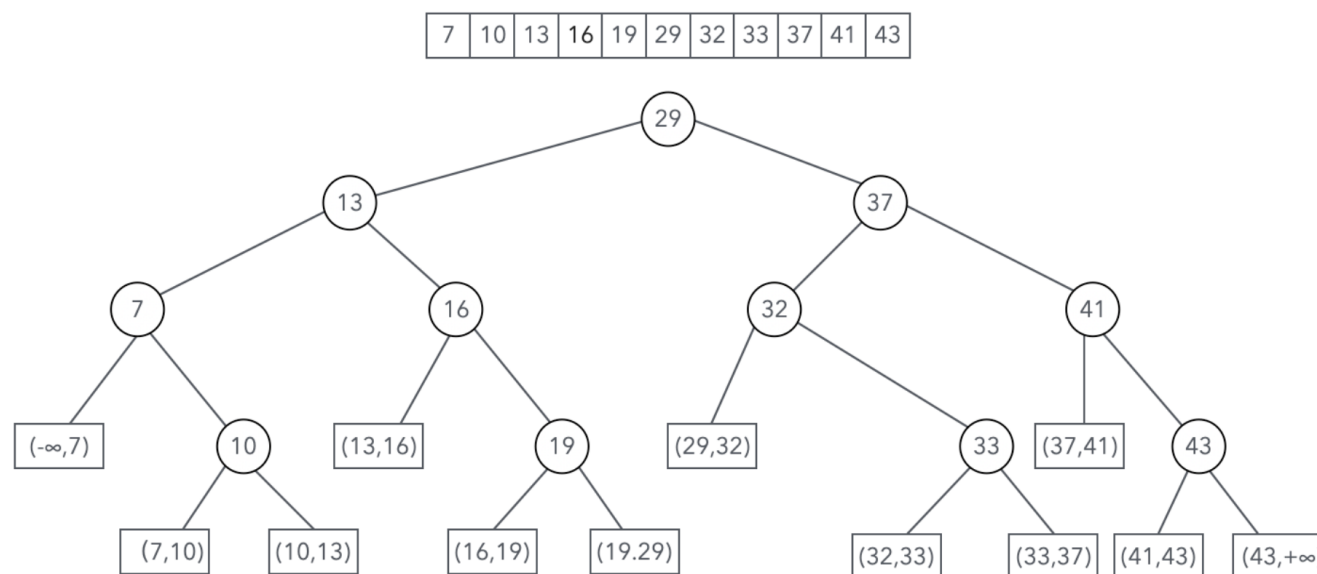
③折半查找可将时间复杂度优化到 $O(\log_2 n)$ ，查找效率较高

④折半查找的最多比较次数 = 折半查找判定树的树高 = 相同结点数量的完全二叉树的高度 = $\lceil \log_2(n+1) \rceil$



考点1 线性表的查找

②折半查找



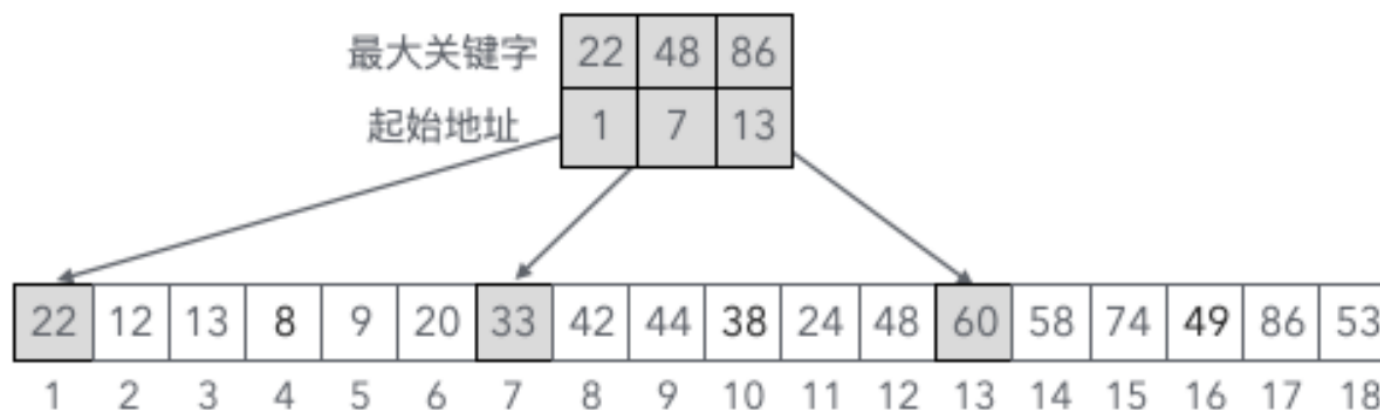
$$ASL_{\text{成功}} = \frac{\text{关键字查找成功比较次数之和}}{\text{查找元素个数}}$$

$$ASL_{\text{失败}} = \frac{\text{关键字查找失败比较次数之和}}{\text{不成功位置个数}}$$



考点1 线性表的查找

③分块查找



①索引表中的每个元素含有各块的最大关键字和各块中第一个元素的地址

②块内元素可以无序，但块间必须有序

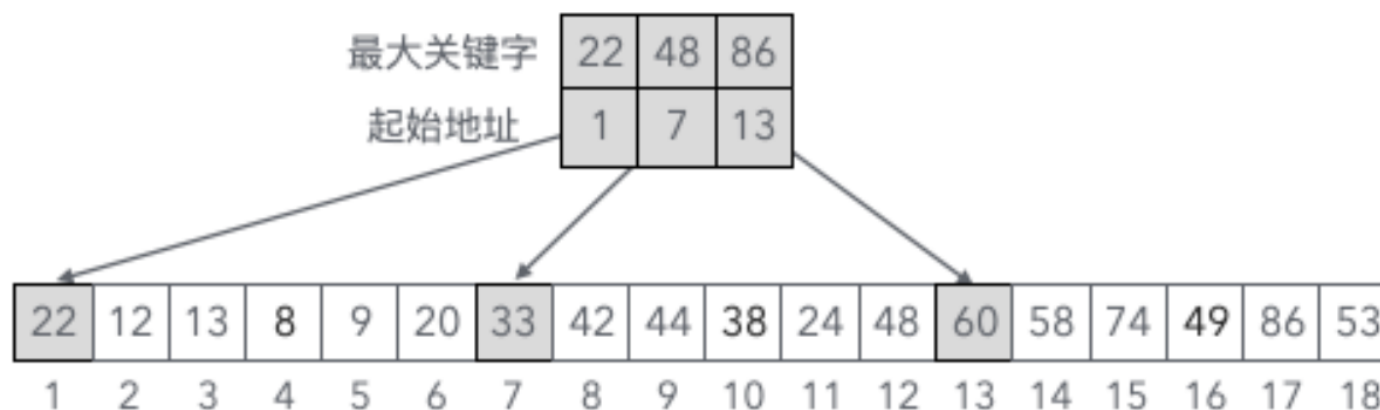
③索引表是有序表，可以进行顺序/折半查找；查找表（块内）是无序的，只能进行顺序查找

④如果既要保证查找效率，又要能满足表中元素动态变化的需求，首选分块查找



考点1 线性表的查找

③分块查找



b 块，每块 s 个记录

$$\text{折半+顺序: ASL}_{\text{成功}} = \frac{\text{关键字查找成功比较次数之和}}{\text{查找元素个数}} + \frac{s+1}{2}$$

$$\text{顺序+顺序: ASL}_{\text{成功}} = \frac{b+1}{2} + \frac{s+1}{2}$$

$\text{ASL}_{\text{失败}}$ 情况较多，一般不讨论



考点1 线性表的查找

【2010】已知一个长度为16的顺序表L，其元素按关键字有序排列，若采用折半查找法查找一个L中不存在的元素，则关键字的比较次数最多是

A. 4

B. 5

C. 6

D. 7



考点1 线性表的查找

【2015】下列选项中，不能构成折半查找中关键字比较序列的是

A. 500, 200, 450, 180

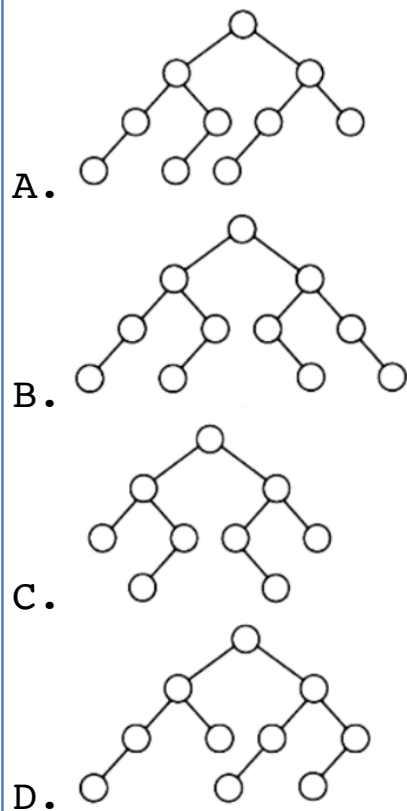
B. 500, 450, 200, 180

C. 180, 500, 200, 450

D. 180, 200, 500, 450

考点1 线性表的查找

【2017】下列二叉树中，可能成为折半查找判定树（不含外部结点）的是





考点1 线性表的查找

【2023】对含600个元素的有序顺序表进行折半查找，关键字间的比较次数最多是

A. 9

B. 10

C. 30

D. 300



考点1 线性表的查找

【2024】下列数据结构中，不适合直接使用折半查找的是

- I. 有序链表
- II. 无序数组
- III. 有序静态链表
- IV. 无序静态链表

- A. 仅I、III
- B. 仅II、IV
- C. 仅II、III、IV
- D. I、II、III、IV**



考点1 线性表的查找

【2013】设包含4个数据元素的集合 $S=\{ 'do', 'for', 'repeat', 'while' \}$ ，各元素的查找概率依次为 $p_1 = 0.35, p_2 = 0.15, p_3 = 0.15, p_4 = 0.35$ 。将 S 保存在一个长度为4的顺序表中，采用折半查找法，查找成功时的平均查找长度为2.2。

- 1) 若采用顺序存储结构保存 S ，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？
- 2) 若采用链式存储结构保存 S ，且要求平均查找长度更短，则元素应如何排列？应使用何种查找方法？查找成功时的平均查找长度是多少？

考点1 线性表的查找

BOK数据结构强化



选择题考频：6/16，分值：2

大题考频：1/16，分值：10

习题训练

1. 设有一个有序表：{2, 5, 8, 12, 17}，使用顺序查找，并且只考虑查找失败的情况。（假设所有失败情况是等概率发生的），则顺序查找失败情况下的平均查找长度（ASL）是

A. 3.00

B. 3.17

C. 3.33

D. 3.50

考点1 线性表的查找

BOK数据结构强化



选择题考频：6/16，分值：2

大题考频：1/16，分值：10

习题训练

2. 在一个长度为12的有序顺序表中每个元素的查找概率相等，则对其进行折半查找时，平均查找长度为

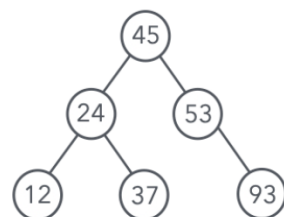
A. $35/12$

B. $31/12$

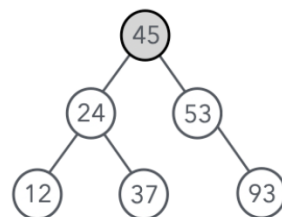
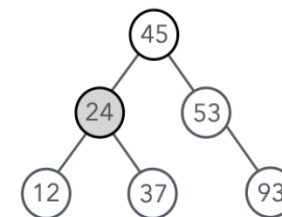
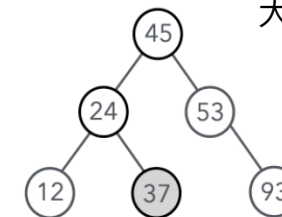
C. $37/12$

D. $39/12$



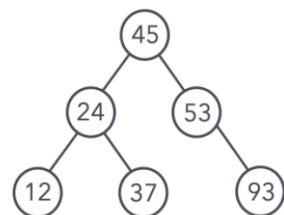


①查找关键字37

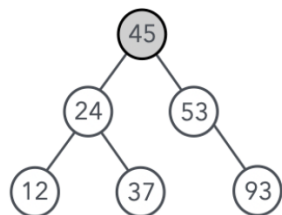
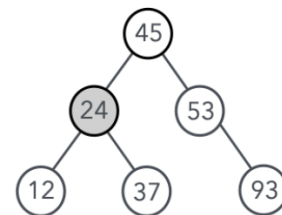
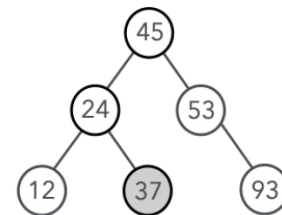
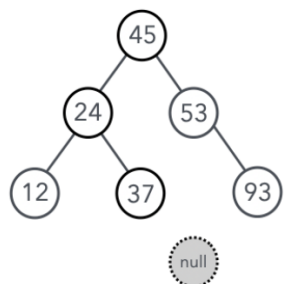
② $37 < 45$, 在左子树中继续查找③ $37 > 24$, 在右子树中继续查找④ $37 = 37$, 查找成功

考点2 BST

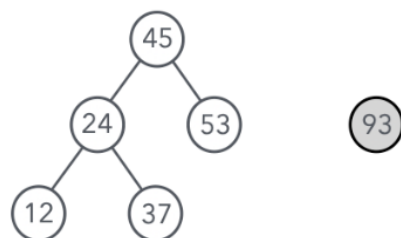
①查找



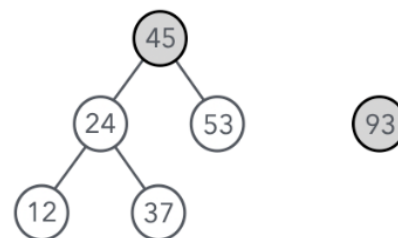
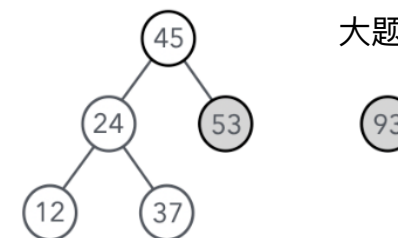
①查找关键字38

② $38 < 45$, 在左子树中继续查找③ $38 > 24$, 在右子树中继续查找④ $38 > 37$, 在右子树中继续查找

⑤二叉排序树为空, 查找失败



①将93插入二叉排序树

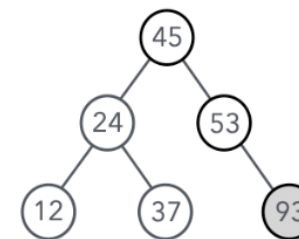
② $93 > 45$, 在右子树中继续查找插入位置③ $93 > 53$, 在右子树中继续查找插入位置

考点2 BST

②插入



④二叉排序树为空, 将93插入(子树的)根节点



⑤完成插入

若BST非空, 则将待插入结点的关键字和根结点的关键字比较
遵循左<根<右的原则, 将结点插入合适的位置

插入基于查找, 所以**新插入的结点一定是叶子结点**
并且是查找不成功时查找路径上访问的最后一个结点的左或右孩子



考点2 BST

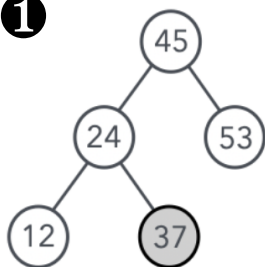
③删除

❶删除叶子结点	直接删除
❷删除结点有左/右孩子	删除后左/右孩子上位
❸删除结点有左右孩子	将删除结点的中序前驱/后继 与删除结点交换 转换为删除叶子结点的情况

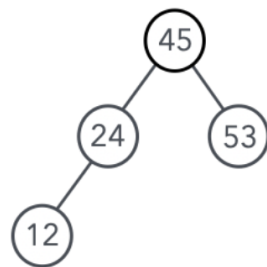


考点2 BST

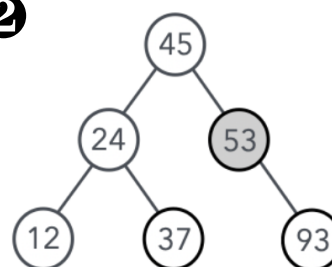
③删除

1

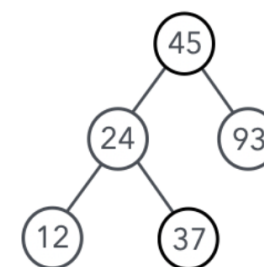
①从二叉排序树删除37



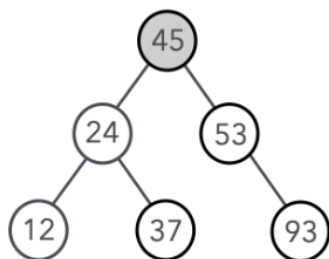
②37为叶子节点, 直接删除

2

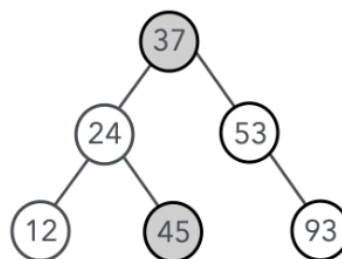
①从二叉排序树删除53



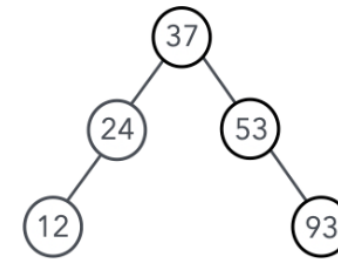
②53有右孩子, 删除后右孩子上位

3

①从二叉排序树删除45



②交换45和45的中序前驱37



③45变为叶子节点, 直接删除



考点2 BST

【2011】对下列关键字序列，不可能构成某二叉排序树中一条查找路径的是

A. 95, 22, 91, 24, 94, 71

B. 92, 20, 91, 34, 88, 35

C. 21, 89, 77, 29, 36, 38

D. 12, 25, 71, 68, 33, 34



考点2 BST

【2013】在任意一棵非空二叉排序树 T_1 中, 删除某结点 v 之后形成二叉排序树 T_2 , 再将 v 插入 T_2 形成二叉排序树 T_3 。下列关于 T_1 与 T_3 的叙述中, 正确的是

I. 若 v 是 T_1 的叶结点, 则 T_1 与 T_3 不同

II. 若 v 是 T_1 的叶结点, 则 T_1 与 T_3 相同

III. 若 v 不是 T_1 的叶结点, 则 T_1 与 T_3 不同

IV. 若 v 不是 T_1 的叶结点, 则 T_1 与 T_3 相同

A. 仅I、III

B. 仅I、IV

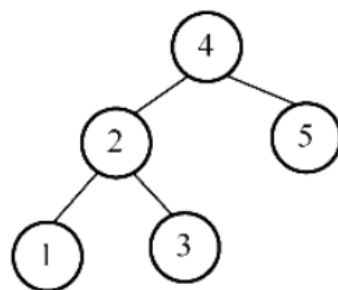
C. 仅II、III

D. 仅II、IV



考点2 BST

【2020】下列给定的关键字输入序列中，不能生成右边二叉排序树的是



A. 4, 5, 2, 1, 3

B. 4, 5, 1, 2, 3

C. 4, 2, 5, 3, 1

D. 4, 2, 1, 3, 5

考点2 BST

BOK数据结构强化



选择题考频: 5/16, 分值: 2

大题考频: 0/16

习题训练

3. 二叉排序树中, 按某种遍历方式遍历可以得到一个有序序列, 则这种遍历方法是

A. 先序

B. 中序

C. 后序

D. 层次

考点2 BST

BOK数据结构强化



选择题考频: 5/16, 分值: 2

大题考频: 0/16

习题训练

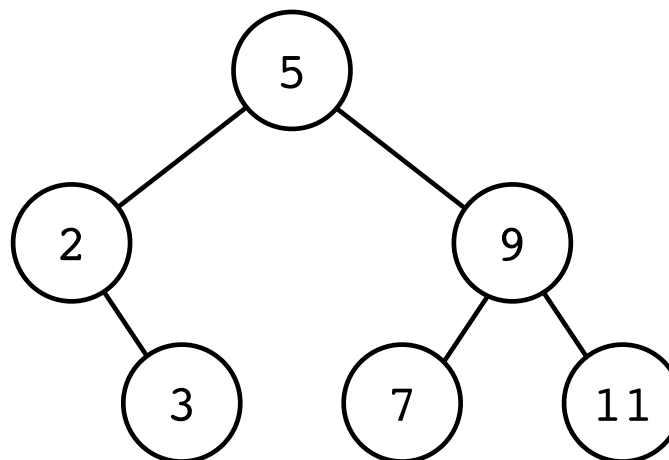
4. 如图所示的二叉排序树, 查找成功时和查找失败时的ASL分别是

A. 3, 6

B. 13/6, 20/7

C. 7/3, 20/7

D. 3, 10/3



考点2 BST

BOK数据结构强化



选择题考频: 5/16, 分值: 2

大题考频: 0/16

习题训练

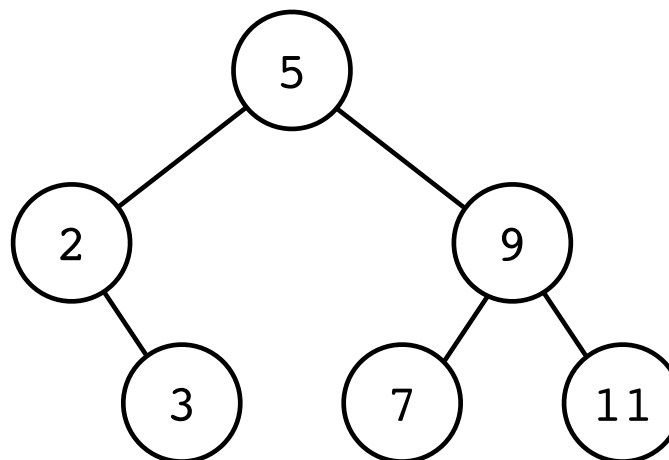
5. 如图所示的二叉排序树, 删除关键字为5的结点后, 根节点的右孩子结点为

A. 3

B. 7

C. 9

D. 11







考点3 AVL

①查找

和BST的规则完全一致



考点3 AVL

②插入

离插入结点**最近且平衡因子绝对值超过1**的祖先结点

以该结点为根的子树称为**最小不平衡子树**

插入后导致AVL不平衡，则调整范围就是最小不平衡子树

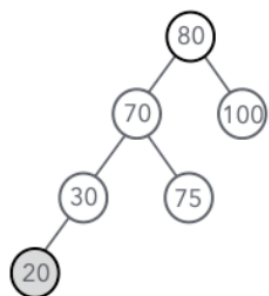
左旋	①右子结点变为旋转结点的父结点
	②右子结点的左子结点变为旋转结点的右子结点
右旋	①左子结点变为旋转结点的父结点
	②左子结点的右子结点变为旋转结点的左子结点



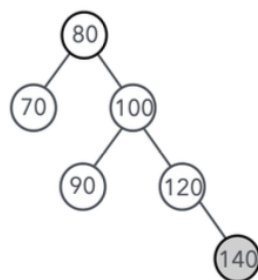
考点3 AVL

②插入

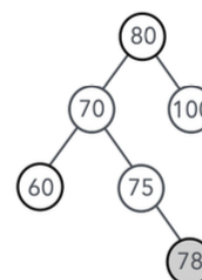
①LL	右旋不平衡结点
②RR	左旋不平衡结点
③LR	先左旋不平衡结点的左孩子, 再右旋不平衡结点
④RL	先右旋不平衡结点的右孩子, 再左旋不平衡结点



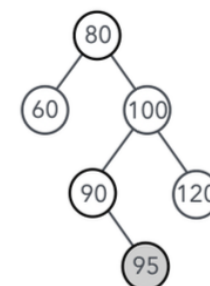
①



②



③



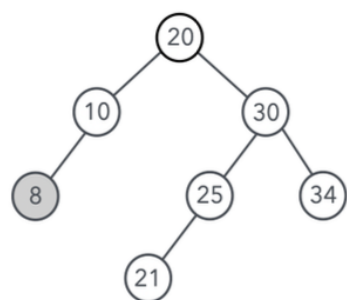
④



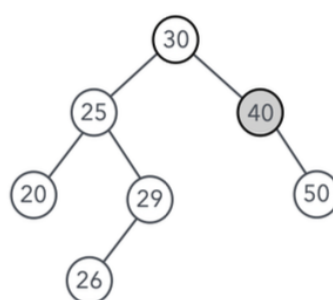
考点3 AVL

③删除

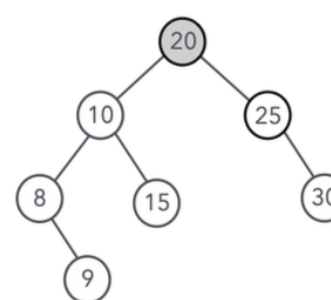
❶删除叶子结点	删除后依旧平衡则直接删除
	删除后不平衡, 则转换为插入情况并调整
❷删除结点有左/右孩子	删除后左/右孩子上位, 转换为插入情况并调整
❸删除结点有左右孩子且删除结点左子树更深	交换删除结点和中序前驱, 转换为插入情况并调整
❹删除结点有左右孩子且删除结点右子树更深	交换删除结点和中序后继, 转换为插入情况并调整



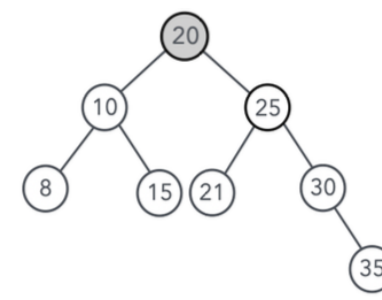
❶



❷



❸



❹



考点3 AVL

④性质

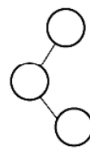
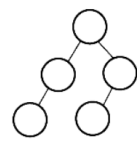
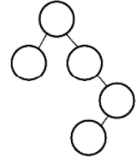
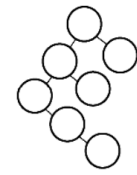
h	1	2	3	4	5	6	7	8
n_h	1	2	4	7	12	20	33	54

- ①假设以 n_h 表示深度为h的平衡二叉树中含有的最少结点数，则 $n_h = n_{h-2} + n_{h-1} + 1$
- ②n个结点的AVL的最小高度为 $\lceil \log_2(n+1) \rceil$
- ③若AVL的高度为h，则树中最多有 $2^h - 1$ 个结点



考点3 AVL

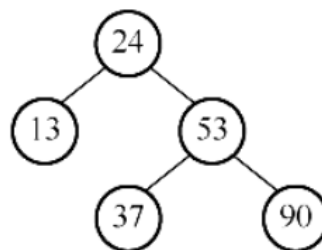
【2009】下列二叉排序树中，满足平衡二叉树定义的是

- A. 
- B. 
- C. 
- D. 



考点3 AVL

【2010】在下图所示的平衡二叉树中插入关键字48后得到一棵新平衡二叉树，在新平衡二叉树中，关键字37所在结点的左、右子结点中保存的关键字分别是



A. 13, 48

B. 24, 48

C. 24, 53

D. 24, 90



考点3 AVL

【2012】若平衡二叉树的高度为6，且所有非叶结点的平衡因子均为1，则该平衡二叉树的结点总数为

A. 12

B. 20

C. 32

D. 33



考点3 AVL

【2013】若将关键字1, 2, 3, 4, 5, 6, 7依次插入初始为空的平衡二叉树T, 则T中平衡因子为0的分支结点的个数是

- A. 0
- B. 1
- C. 2
- D. 3**



考点3 AVL

【2015】现有一颗无重复关键字的平衡二叉树（AVL），对其进行中序遍历可得到一个降序序列。下列关于该平衡二叉树的叙述中，正确的是

- A. 根结点的度一定为2
- B. 树中最小元素一定是叶结点
- C. 最后插入的元素一定是叶结点
- D. 树中最大元素一定是无左子树**



考点3 AVL

【2019】在任意一棵非空平衡二叉树 (AVL树) T_1 中, 删除某结点 v 之后形成平衡二叉树 T_2 , 再将 v 插入 T_2 形成平衡二叉树 T_3 。下列关于 T_1 与 T_3 的叙述中, 正确的是

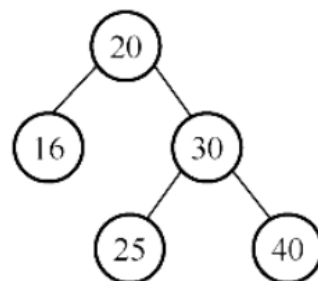
- I. 若 v 是 T_1 的叶结点, 则 T_1 与 T_3 可能不相同
- II. 若 v 不是 T_1 的叶结点, 则 T_1 与 T_3 一定不相同
- III. 若 v 不是 T_1 的叶结点, 则 T_1 与 T_3 一定相同

- A. 仅I
- B. 仅II
- C. 仅I、II
- D. 仅I、III



考点3 AVL

【2021】给定平衡二叉树如下图所示，插入关键字23后，根中的关键字是

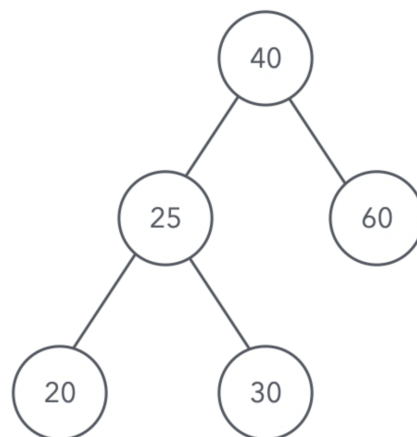


- A. 16
- B. 20
- C. 23
- D. 25**



习题训练

6. 在下图所示的平衡二叉树中插入关键字29后，得到一棵新的平衡二叉树。在新平衡二叉树中，关键字25所在结点的左、右子结点中保存的关键字分别是



A. 20, 30

B. 20, 29

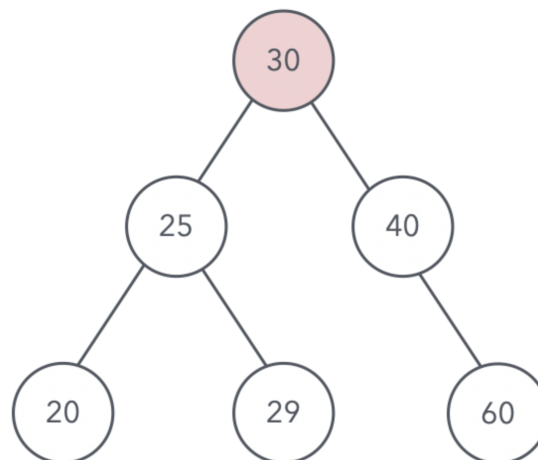
C. 29, 30

D. 20, 60



习题训练

7. 现有平衡二叉树如下，删除30节点后，节点25的右孩子会变成



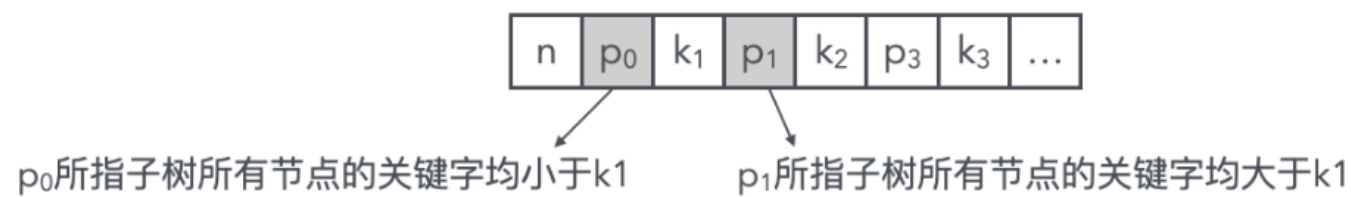
- A. 29
- B. 40
- C. 60
- D. 空





考点4 B树和B+树

①结点结构



n 为结点内关键字的个数（一般省略）

k_i 为关键字

p_i 为指向子树根结点的指针



考点4 B树和B+树

②性质

	B树	B+树
关键字和子树的关系	n个关键字对应n+1棵子树	n个关键字对应n棵子树
根结点子树	[2 , m]	[2 , m]
根结点关键字	[1 , m-1]	
非根结点子树	[⌈ m / 2 ⌋ , m]	[⌈ m / 2 ⌋ , m]
非根结点关键字	[⌈ m / 2 ⌋ - 1 , m-1]	
叶子结点/失败结点/终端结点	所有叶子结点都在同一层	
	若认为叶子结点是外部结点 则叶子结点且不带信息 (可类比折半查找判定树的失败结点)	叶子结点中包含了所有关键字信息 以及指向含这些关键字记录的指针 叶子结点本身按照关键字大小 从小到大顺序链接
查找方式	随机查找	随机查找和顺序查找
高度	$log_m(n+1) \leq h \leq log_{\lceil \frac{m}{2} \rceil}(\frac{n+1}{2}) + 1$	



考点4 B树和B+树

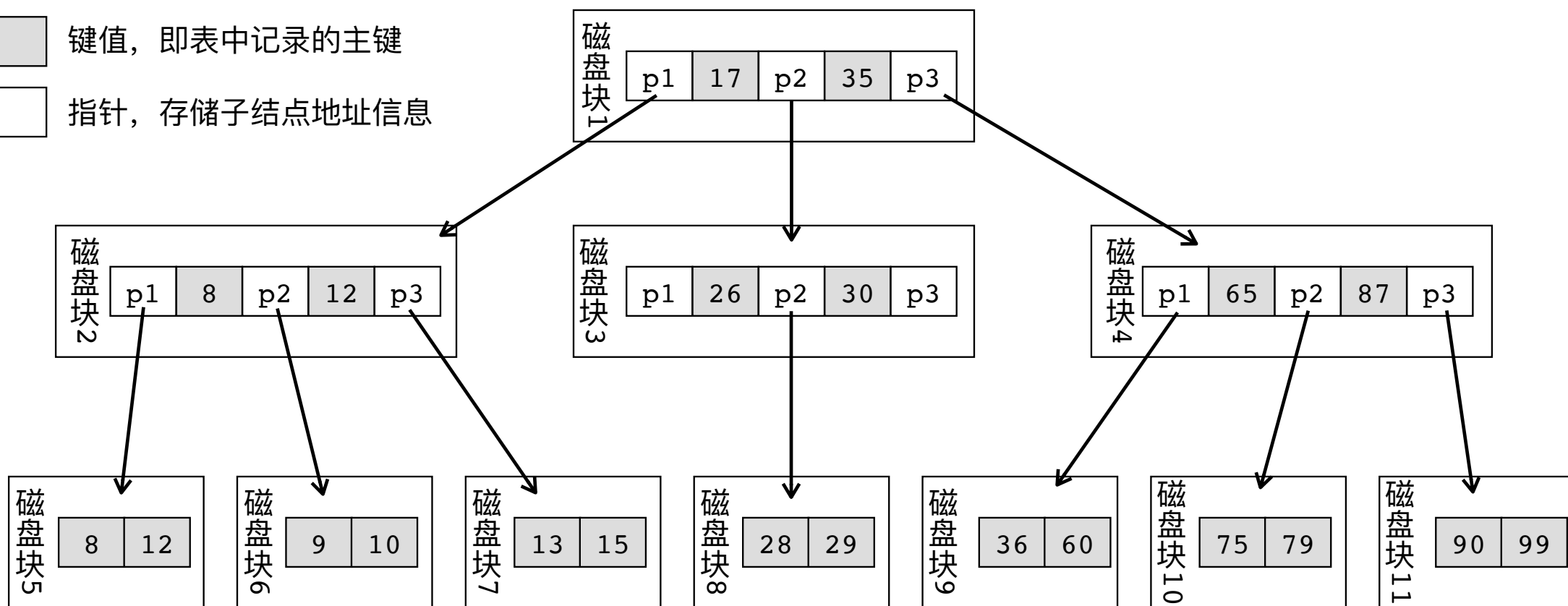
③查找



键值，即表中记录的主键



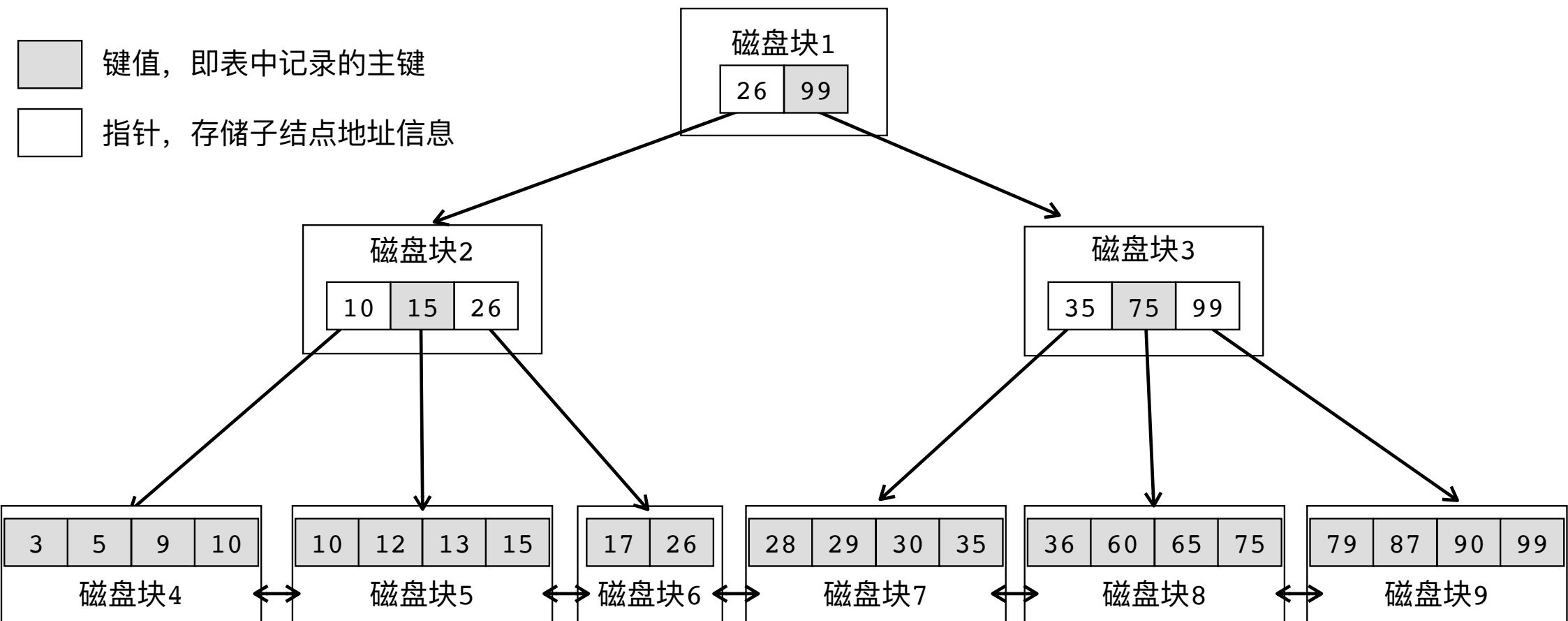
指针，存储子结点地址信息





考点4 B树和B+树

③查找





考点4 B树和B+树

④插入

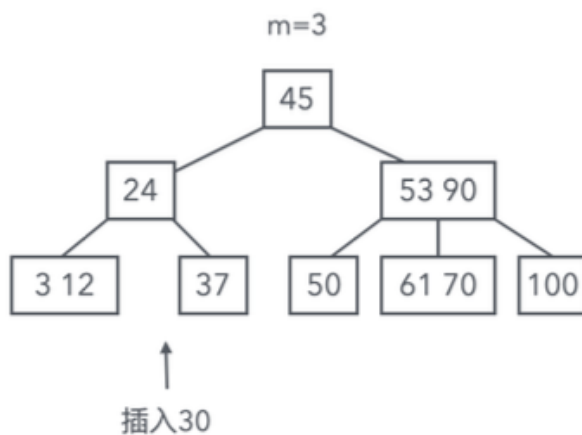
B树

①插入后结点关键字个数 $\leq m-1$

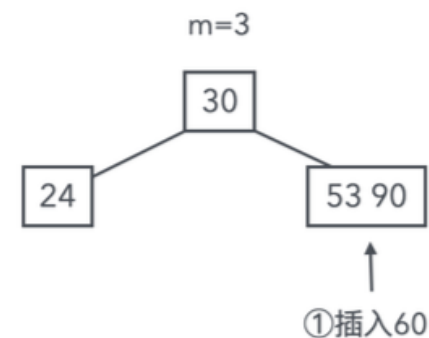
直接插入

②插入后结点关键字个数 $> m-1$

结点（插入后的）第 $\lceil m/2 \rceil$ 个关键字进入父结点
原结点一分为二



①



②

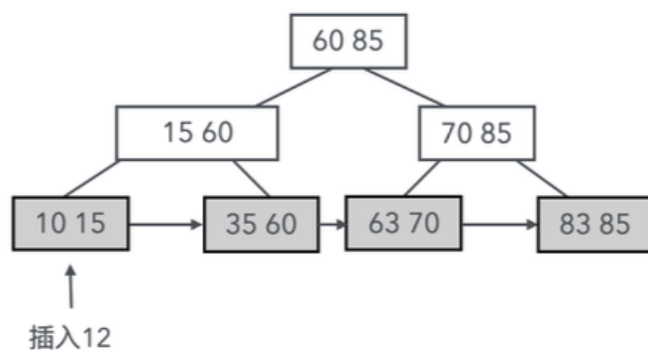


考点4 B树和B+树

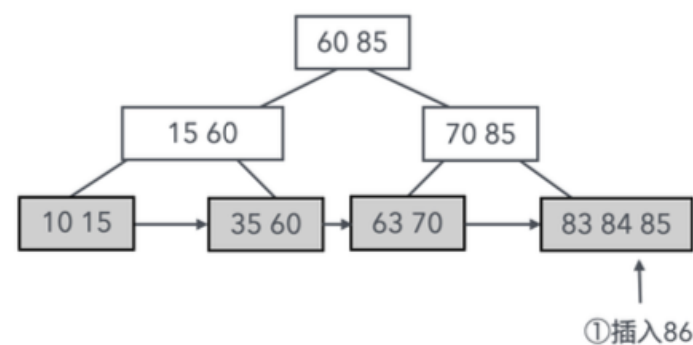
④插入

B+树

①插入后结点关键字个数 $\leq m$	直接插入
②插入后结点关键字个数 $> m$	从第 $\lceil m/2 \rceil$ 个关键字开始 原结点一分为二
插入过程中要时刻注意是否需要修改索引中的关键字	



①



②

tips: 为了方便叙述，研究删除所讨论的终端结点指的都是最下排的一层含关键字的结点
而不是不含任何信息的叶子结点/外部结点



考点4 B树和B+树

B树

⑤删除

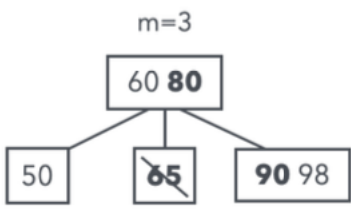
①被删关键字不在终端结点	用被删关键字的前驱/后继关键字代替被删关键字的位置 转化为删除终端结点关键字的情况	
②被删关键字在终端结点	②-1 删除后关键字个数 $\geq \lceil m/2 \rceil - 1$	直接删除
	②-2 删除后关键字 $< \lceil m/2 \rceil - 1$ 且兄弟够借	父下子上
	②-3 删除后关键字 $< \lceil m/2 \rceil - 1$ 且兄弟不够借	父下与子合并



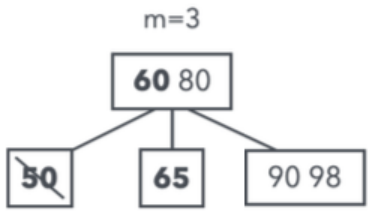
①



②-1



②-2



②-3

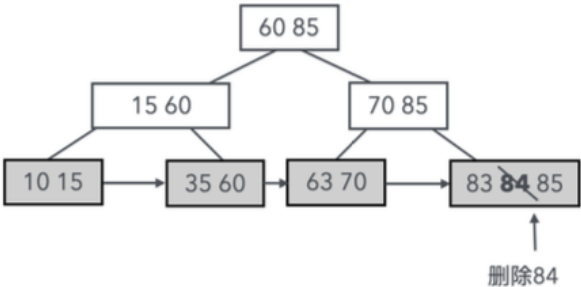
tips: 为了方便叙述，研究删除所讨论的终端结点指的都是最下排的一层含关键字的结点
而不是不含任何信息的叶子结点/外部结点

考点4 B树和B+树

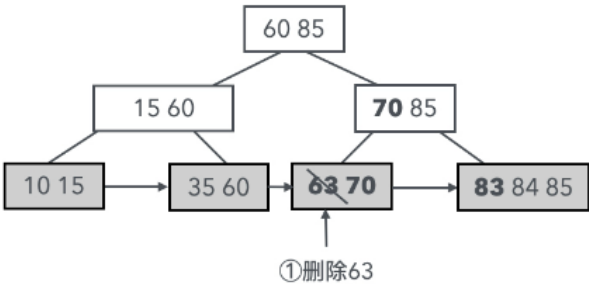
B+树

⑤删除

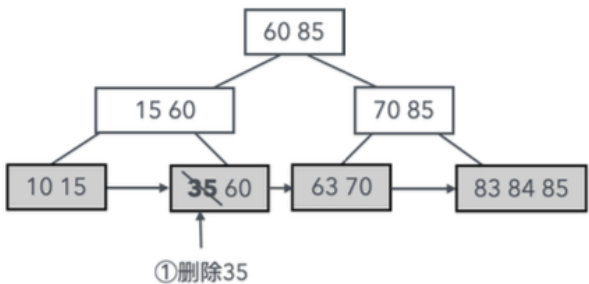
❶被删关键字在终端结点	❶-1 删除后关键字个数 $\geq \lceil m/2 \rceil$	直接删除
	❶-2 删除后关键字 $< \lceil m/2 \rceil$ 且兄弟够借	兄弟结点的关键字进入当前结点
	❶-3 删除后关键字 $< \lceil m/2 \rceil$ 且兄弟不够借	当前结点与兄弟结点合并
删除过程中要时刻保持终端结点关键字和非终端结点关键字的一致性		



❶-1



❶-2



❶-3



考点4 B树和B+树

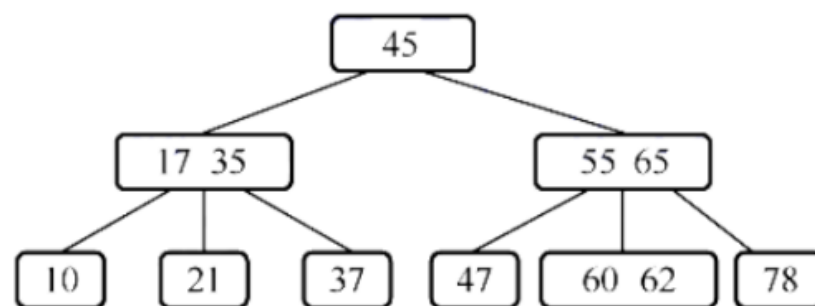
【2009】下列叙述中，不符合m阶B树定义要求的是

- A. 根结点至多有m棵子树
- B. 所有叶结点都在同一层上
- C. 各结点内关键字均升序或降序排列
- D. 叶结点之间通过指针链接**



考点4 B树和B+树

【2012】已知一棵3阶B树，如下图所示。删除关键字78得到一棵新B树，其最右叶结点中的关键字是



- A. 60
- B. 60, 62
- C. 62, 65
- D. 65**



考点4 B树和B+树

【2013】在一棵高度为2的5阶B树中, 所含关键字的个数至少是

A. 5

B. 7

C. 8

D. 14



考点4 B树和B+树

【2014】在一棵有15个关键字的4阶B树中，含关键字的结点个数最多是

- A. 5
- B. 6
- C. 10
- D. 15**



考点4 B树和B+树

【2016】B+树不同于B树的特点之一是

A. 能支持顺序查找

B. 结点中含有关键字

C. 根结点至少有两个分支

D. 所有叶结点都在同一层上



考点4 B树和B+树

【2017】下列应用中, 适合使用B+树的是

- A. 编译器中的词法分析
- B. 关系数据库中的索引**
- C. 网络中的路由表快速查找
- D. 操作系统的磁盘空闲块管理



考点4 B树和B+树

【2018】高度为5的3阶B树含有的关键字个数至少是

A. 15

B. 31

C. 62

D. 242



考点4 B树和B+树

【2020】依次将关键字5, 6, 9, 13, 8, 2, 12, 15插入初始为空的4阶B树后, 根结点中包含的关键字是

A. 8

B. 6, 9

C. 8, 13

D. 9, 12



考点4 B树和B+树

【2021】在一棵高度为3的3阶B树中，根为第1层，若第2层中有4个关键字，则该树的结点数最多是

A. 11

B. 10

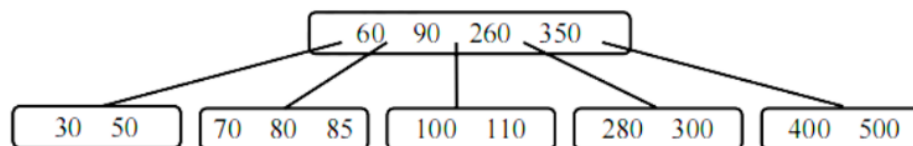
C. 9

D. 8



考点4 B树和B+树

【2022】在下图所示的5阶B树T中，删除关键字260之后需要进行必要的调整，得到新的B树T1。下列选项中，不可能是T1根结点中关键字序列的是



- A. 60, 90, 280
- B. 60, 90, 350
- C. 60, 85, 110, 350
- D. 60, 90, 110, 350**





考点5 散列表

①名词区分

同义词	具有相同散列函数值的关键字
非同义词	具有不同散列函数值的关键字
冲突/碰撞	不同的关键字映射到了同一散列地址的现象
堆积/二次聚集	用线性探测法处理同义词的冲突过程中又添加了非同义词的冲突
装填因子	装填因子 $\alpha = n/m = \text{表中关键字个数} / \text{散列表长度}$



考点5 散列表

①名词区分

构造散列函数的方法	直接定址法	
	除留余数法	
	数字分析法	
	平方取中法	
处理冲突的方法	开放定址法	线性探测法
		平方探测法
		双散列法
		伪随机序列法
	链地址法	



考点5 散列表

②处理冲突的方法

线性探测法

- ①冲突发生时，顺序查看表中下一个单元，直到找出一个空闲单元或查遍全表
- ②探测到表尾时，下一个探测地址是表头
- ③处理同义词的冲突过程中可能又添加了非同义词的冲突
- ④不可以直接删除散列表中关键字，仅做删除标记

$H(\text{key}) = \text{key} \% 10$ ，将{10, 21, 32, 42, 52, 75}插入散列表中

0	1	2	3	4	5	6	7	8	9	10	11	12



考点5 散列表

②处理冲突的方法

线性探测法

- ①冲突发生时, 顺序查看表中下一个单元, 直到找出一个空闲单元或查遍全表
- ②探测到表尾时, 下一个探测地址是表头
- ③处理同义词的冲突过程中可能又添加了非同义词的冲突
- ④不可以直接删除散列表中关键字, 仅做删除标记

$H(\text{key}) = \text{key} \% 10$, 将{10, 21, 32, 42, 52, 75}插入散列表中

10	21	32	42	52	75								
----	----	----	----	----	----	--	--	--	--	--	--	--	--

0 1 2 3 4 5 6 7 8 9 10 11 12

$$10 \% 10 = 0$$

$$21 \% 10 = 1$$

$$32 \% 10 = 2$$

$$42 \% 10 = 2, \text{冲突}, H1 = (2+1) \% 13 = 3$$

$$52 \% 10 = 2, \text{冲突}, H1 = (2+1) \% 13 = 3, \text{冲突}, H2 = (2+2) \% 13 = 4$$

$$75 \% 10 = 5$$



线性探测法

考点5 散列表

②处理冲突的方法

$H(\text{key}) = \text{key} \% 10$ ，将{10, 21, 32, 42, 52, 75}插入散列表中

10	21	32	42	52	75							
0	1	2	3	4	5	6	7	8	9	10	11	12

这三个地址是映射不到的
所以分析ASL_{失败}时不需要考虑

$$10 \% 10 = 0$$

$$21 \% 10 = 1$$

$$32 \% 10 = 2$$

$$42 \% 10 = 2, \text{冲突}, H_1 = (2+1) \% 13 = 3$$

$$52 \% 10 = 2, \text{冲突}, H_1 = (2+1) \% 13 = 3, \text{冲突}, H_2 = (2+2) \% 13 = 4$$

$$75 \% 10 = 5$$

关键字	10	21	32	42	52	75
比较次数	1	1	1	2	3	1

$$ASL_{\text{成功}} = (1+1+1+2+3+1) / 6 = 1.5$$

散列地址	0	1	2	3	4	5	6	7	8	9
比较次数	7	6	5	4	3	2	1	1	1	1

$$ASL_{\text{失败}} = (7+6+5+4+3+2+1+1+1+1) / 10 = 3.1$$



考点5 散列表

②处理冲突的方法

平方探测法

- ① $H_i = (H(\text{key}) + d_i) \% m$, $d_i = 1^2, -1^2, 2^2, -2^2, \dots$ (公式可能有变动, 以题目为准)
- ② 散列表长度 m 必须是一个可以表示成 $4k+3$ 的素数
- ③ 可以避免堆积, 但无法探测到散列表上的所有单元
- ④ 不可以直接删除散列表中关键字, 仅做删除标记

$H(\text{key}) = \text{key} \% 11$, 将{1, 13, 12, 34, 38, 33, 27, 22}插入散列表中

0	1	2	3	4	5	6	7	8	9	10



考点5 散列表

②处理冲突的方法

平方探测法

- ① $H_i = (H(\text{key}) + d_i) \% m$, $d_i = 1^2, -1^2, 2^2, -2^2, \dots$ (公式可能有变动, 以题目为准)
- ② 散列表长度 m 必须是一个可以表示成 $4k+3$ 的素数
- ③ 可以避免堆积, 但无法探测到散列表上的所有单元
- ④ 不可以直接删除散列表中关键字, 仅做删除标记

$H(\text{key}) = \text{key} \% 11$, 将{1, 13, 12, 34, 38, 33, 27, 22}插入散列表中

12	1	13		27	34	38	22			33
0	1	2	3	4	5	6	7	8	9	10

$$1 \% 11 = 1$$

$$13 \% 11 = 2$$

$$12 \% 11 = 1, \text{冲突}, H_1 = (1+1) \% 11 = 2, \text{冲突}, H_2 = (1-1) \% 11 = 0$$

$$34 \% 11 = 1, \text{冲突}, H_1 = (1+1) \% 11 = 2, \text{冲突}, H_2 = (1-1) \% 11 = 0, \text{冲突}, H_3 = (1+4) \% 11 = 3 = 5$$

$$38 \% 11 = 5, \text{冲突}, H_1 = (5+1) \% 11 = 6$$

$$33 \% 11 = 0, \text{冲突}, H_1 = (0+1) \% 11 = 1, \text{冲突}, H_2 = (0-1) \% 11 = 10$$

$$27 \% 11 = 5, \text{冲突}, H_1 = (5+1) \% 11 = 6, \text{冲突}, H_2 = (5-1) \% 11 = 4$$

$$22 \% 11 = 0, \text{冲突}, H_1 = (0+1) \% 11 = 1, \text{冲突}, H_2 = (0-1) \% 11 = 10, \text{冲突}, H_3 = (0+4) \% 11 = 4, \text{冲突}, H_4 = (0-4) \% 11 = 7$$



平方探测法

考点5 散列表

②处理冲突的方法

$H(\text{key}) = \text{key} \% 11$, 将{1, 13, 12, 34, 38, 33, 27, 22}插入散列表中

12	1	13		27	34	38	22			33
0	1	2	3	4	5	6	7	8	9	10

$$1 \% 11 = 1$$

$$13 \% 11 = 2$$

$$12 \% 11 = 1, \text{冲突}, H1 = (1+1) \% 11 = 2, \text{冲突}, H2 = (1-1) \% 11 = 0$$

$$34 \% 11 = 1, \text{冲突}, H1 = (1+1) \% 11 = 2, \text{冲突}, H2 = (1-1) \% 11 = 0, \text{冲突}, H3 = (1+4) \% 11 = 3 = 5$$

$$38 \% 11 = 5, \text{冲突}, H1 = (5+1) \% 11 = 6$$

$$33 \% 11 = 0, \text{冲突}, H1 = (0+1) \% 11 = 1, \text{冲突}, H2 = (0-1) \% 11 = 10$$

$$27 \% 11 = 5, \text{冲突}, H1 = (5+1) \% 11 = 6, \text{冲突}, H2 = (5-1) \% 11 = 4$$

$$22 \% 11 = 0, \text{冲突}, H1 = (0+1) \% 11 = 1, \text{冲突}, H2 = (0-1) \% 11 = 10, \text{冲突}, H3 = (0+4) \% 11 = 4, \text{冲突}, H4 = (0-4) \% 11 = 7$$

关键字	1	13	12	34	38	33	27	22
比较次数	1	1	3	4	2	3	3	5

$$ASL_{\text{成功}} = (1+1+3+4+2+3+3+5) / 8 = 11/4$$

散列地址	0	1	2	3	4	5	6	7	8	9	10
比较次数	6	5	2	1	3	4	7	2	1	1	3

$$ASL_{\text{失败}} = (6+5+2+1+3+4+7+2+1+1+3) / 11 = 35/11$$

以散列地址0为例, 查找的地址次序是: 0, 1, 10, 4, 7, 9, 直到9才访问到空单元, 因此查找失败的比较次数是6次

②处理冲突的方法

- ①将所有的同义词存储在一个链表中，这个链表由其散列地址唯一标识
- ②可以直接删除关键字

Diagram illustrating a B-tree structure with 13 slots (0 to 12). The slots contain pointers to leaf nodes (represented as arrays of values and a null pointer ^).

- Slot 1 points to a leaf node containing [14, 1, 27, 79, ^].
- Slot 3 points to a leaf node containing [68, 55, ^].
- Slot 6 points to a leaf node containing [19, 84, ^].
- Slot 7 points to a leaf node containing [20, ^].
- Slot 10 points to a leaf node containing [23, 10, ^].
- Slot 11 points to a leaf node containing [11, ^].

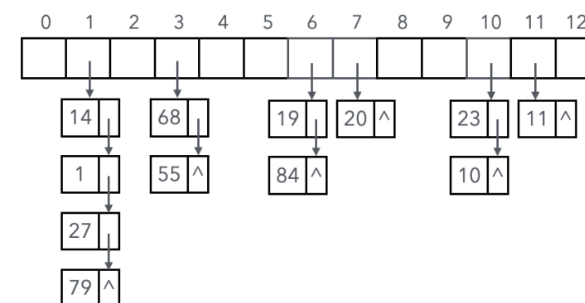


链地址法

考点5 散列表

②处理冲突的方法

$H(key) = key \% 13$



关键字	14	1	27	79	68	55	19	84	20	23	10	11
比较次数	1	2	3	4	1	2	1	2	1	1	2	1

$$ASL_{成功} = (1+2+3+4+1+2+1+2+1+1+2+1)/12 = 1.75$$

散列地址	0	1	2	3	4	5	6	7	8	9	10	11	12
比较次数	1	5	1	3	1	1	3	2	1	1	3	2	1

$$ASL_{失败} = (1+5+1+3+1+1+3+2+1+1+3+2+1)/13 = \frac{25}{13} = 1.923$$

(空指针计入比较次数)



考点5 散列表

③影响查找效率的因素

①装填因子 α

① $\alpha = n/m =$ 表中关键字个数/散列表长度

② α 定义了一个散列表的装满程度， α 越大，产生冲突可能性越大

③平均查找长度ASL与n或m无直接关系，而是依赖于装填因子 α

②散列函数：散列函数设计得越好，关键字分布越均匀，产生冲突的可能性越小，查找路径越短

③处理冲突的方法：决定了发生冲突后查找的效率，有些方法可能还会引入新的冲突，造成查找距离变长



考点5 散列表

【2011】为提高散列表的查找效率，可以采取的正确措施是

I. 增大装填（载）因子

II. 设计冲突（碰撞）少的散列函数

III. 处理冲突（碰撞）时避免产生聚集（堆积）现象

A. 仅I

B. 仅II

C. 仅I、II

D. 仅II、III



考点5 散列表

【2014】用哈希（散列）方法处理冲突（碰撞）时可能出现堆积（聚集）现象，下列选项中，会受堆积现象直接影响的是

- A. 存储效率
- B. 散列函数
- C. 装填（装载）因子
- D. 平均查找长度**



考点5 散列表

【2018】现有长度为7、初始为空的散列表HT，散列函数 $H(k) = k \% 7$ ，用线性探测再散列法解决冲突。将关键字22，43，15依次插入HT后，查找成功的平均查找长度是

A. 1.5

B. 1.6

C. 2

D. 3



考点5 散列表

【2023】现有长度为5、初始为空的散列表HT，散列函数 $H(k) = (k+4) \% 5$ ，用线性探查再散列法解决冲突。若将关键字序列2022，12，25依次插入HT，然后删除关键字25，则HT中查找失败的平均查找长度为

- A. 1
- B. 1.6
- C. 1.8**
- D. 2.2



考点5 散列表

【2024】将关键字序列20, 3, 11, 18, 9, 14, 7依次存储到初始为空、长度为11的散列表HT中，散列函数 $H(\text{key}) = (\text{key} * 3) \% 11$ 。 $H(\text{key})$ 计算出的初始散列地址为 H_0 ，发生冲突时探查地址序列是 $H_1, H_2, H_3, \dots$ ，其中， $H_k = (H_0 + k^2) \% 11$, $k=1, 2, 3, \dots$ 。

请回答下列问题：

- 1) 画出所构造的HT，并计算HT的装填因子。
- 2) 给出在HT中查找关键字14的关键字比较序列。
- 3) 在HT中查找关键字8，确认查找失败时的散列地址是多少？

考点5 散列表

BOK数据结构强化



选择题考频：6/16，分值：2

大题考频：2/16，分值：10

习题训练

8. 散列表长 $m=14$ ， $\text{Hash}(\text{key})=\text{key}\%11$ ，若插入关键字15、38、61、84，并使用二次探测法解决冲突，则49的散列地址是

A. 8

B. 3

C. 5

D. 9

考点5 散列表

BOK数据结构强化



选择题考频：6/16，分值：2

大题考频：2/16，分值：10

习题训练

9. 下列关于散列表的说法中，正确的是
- A. 散列表的平均查找长度与处理冲突的方法无关
 - B. 散列表中，比较操作是不可避免的**
 - C. 散列表查找成功的平均查找长度与表长有关
 - D. 若删除一个元素，可以直接删除





考点6 红黑树

①定义与性质

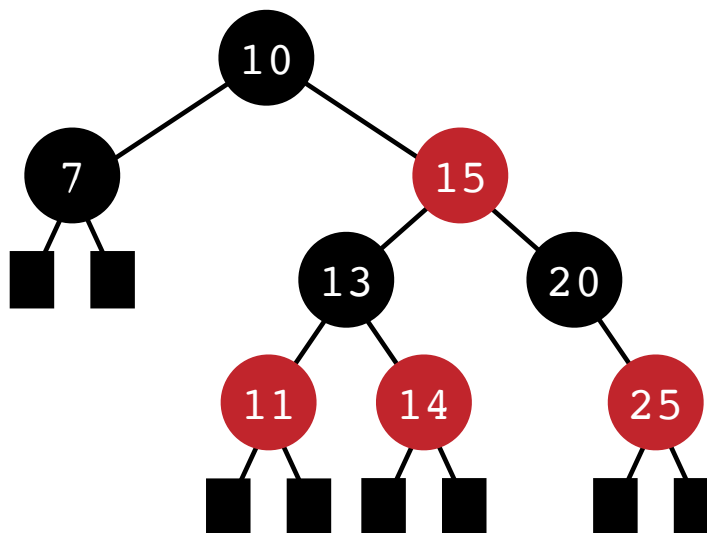
- ①结点是红色或黑色，根结点和叶子结点必是黑色
- ②每个红结点必须有两个黑色的子结点，即不能出现两个连续的红结点
- ③从任一结点到每个叶子结点的所有路径都包含相同数目的黑色结点
- ④从某结点出发（不含该结点）到达任一叶子结点（包含叶结点）的路径上的黑结点总数即为黑高bh



考点6 红黑树

①定义与性质

- ①根结点的黑高 $\geq h/2$
- ②从根到叶结点的最长路径 $\leq$ 最短路径的2倍
最短路径上只有黑结点
最长路径中红结点的个数 = 黑结点的个数, 交替出现
- ③有 n 个内部结点的红黑树的高度 $h \leq 2\log_2(n+1)$





- ①插入的新结点一定是红结点
- ②染色就是红黑互换

考点6 红黑树

②插入

①空树	插入，染色	
②父黑	直接插入	
③父红且叔红	叔父染色，祖父染色	
	若祖父是根节点，祖父染色，否则将祖父看作新结点，重新判断类型并调整	
④父红且叔黑	④-1 LL	祖父 右旋 ，父亲、祖父染色
	④-2 LR	父亲 左旋 ，父亲看作新结点，重新判断类型并调整
	④-3 RR	祖父 左旋 ，父亲、祖父染色
	④-4 RL	父亲 右旋 ，父亲看作新结点，重新判断类型并调整



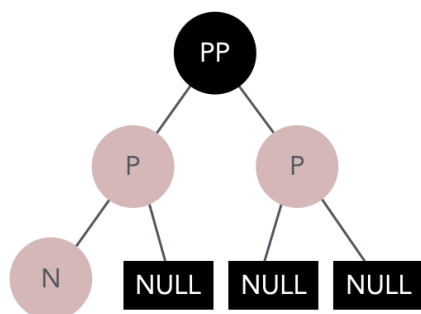
①插入的新结点一定是红结点

②染色就是红黑互换

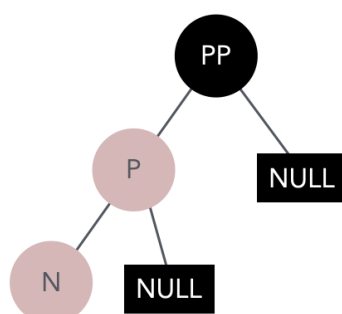
考点6 红黑树

②插入

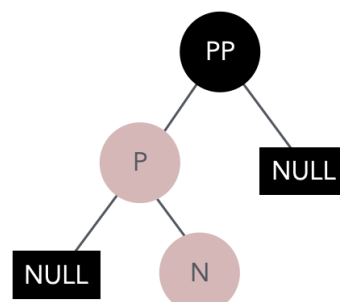
③父红且叔红	叔父染色，祖父染色	
	若祖父是根节点，祖父染色，否则将祖父看作新结点，重新判断类型并调整	
④父红且叔黑	④-1 LL	祖父右旋，父亲、祖父染色
	④-2 LR	父亲左旋，父亲看作新结点，重新判断类型并调整
	④-3 RR	祖父左旋，父亲、祖父染色
	④-4 RL	父亲右旋，父亲看作新结点，重新判断类型并调整



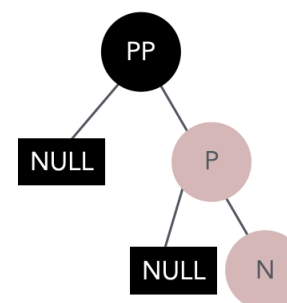
③



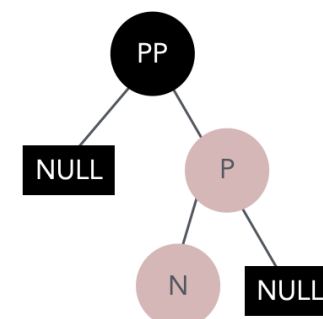
④-1



④-2



④-3



④-4



考点6 红黑树

③红黑树 vs AVL

- ①设计红黑树的目的就是解决AVL维护起来比较麻烦的问题，红黑树的读取略逊于AVL，维护强于AVL，每次插入和删除的平均旋转次数远小于AVL
- ②AVL追求绝对严格的平衡，必须满足左右子树高度差不超过1，而红黑树放弃追求完全平衡，它的旋转次数少，**插入最多2次旋转，删除最多3次旋转**，所以对于搜索、插入、删除操作较多的情况下，红黑树的效率是优于AVL的

考点6 红黑树

BOK数据结构强化



选择题考频: 0/16

大题考频: 0/16

习题训练

10. 在关于红黑树和AVL的说法中, 正确的是

- A. 红黑树查找比AVL快
- B. 红黑树插入和删除时旋转次数比AVL树多
- C. 红黑树的结构比AVL更平衡
- D. 红黑树和AVL的插入、删除时间复杂度都是 $O(\log_2 n)$**

考点6 红黑树

BOK数据结构强化

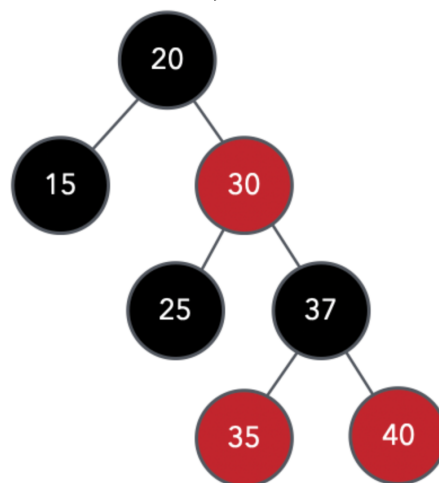


选择题考频: 0/16

大题考频: 0/16

习题训练

11. 下图所示红黑树，插入节点45并调整完毕后，图中有多少个黑节点（不包括NULL）



- A. 3
- B. 4
- C. 5**
- D. 6



祝你今天特别特别开心^•u-^